Reflective Journal: ITAI 1371 Midterm Project

For this midterm project, my group worked with the Telco Customer Churn dataset. The main goal was to clean and prepare the dataset so it could eventually be used to train machine learning models that predict whether a customer is likely to cancel their service. My part of the project focused on one-hot encoding the categorical data, scaling important numerical columns, comparing the data before and after preprocessing, and using class balancing with Logistic Regression.

One thing this project helped me understand better is that raw data usually cannot be placed directly into a machine learning model. The Telco dataset had a lot of useful information, but most of the columns contained text values such as contract type, payment method, internet service, and whether the customer had different services. These values make sense to a person, but the model needs numbers to work with them. My first responsibility was to make those categories cleaner and easier to encode.

Some columns contained values like “No internet service” or “No phone service.” Technically, those values were different from a regular “No,” but they were still telling us that the customer did not have that service. I changed those values into a simpler “No” category using Python. This made the categories more consistent and prevented the encoding process from creating extra columns that were not really necessary. It also showed me that data cleaning involves thinking about what the values actually mean instead of only changing them because the instructions say to.

After cleaning the categories, I used one-hot encoding to convert the remaining categorical columns into numeric columns containing 0s and 1s. This was one of the more important parts of my section because the dataset had many object or text columns. One-hot encoding allowed the model to use information like contract type and payment method without incorrectly treating the categories as if they had a numerical ranking. For example, a month-to-month contract is not mathematically “less than” a two-year contract, so assigning them random numbers could give the model the wrong idea.

I also applied `StandardScaler` to `tenure`, `MonthlyCharges`, and `TotalCharges`. These columns were all numerical, but their values were on different ranges. Tenure was measured in months, while total charges could reach several thousand dollars. Without scaling, a model like Logistic Regression could be influenced more by a column simply because its numbers were larger. Standardizing the columns placed them on a more comparable scale.

The before-and-after graphs were helpful because they showed what scaling actually does. The general shape of each distribution stayed similar, but the values were moved onto a standardized scale. Before this assignment, I understood scaling mostly as a formula or code command. Seeing the graphs made it clearer that scaling does not erase the original pattern in the data. It changes the range and center of the values so the model can work with them more fairly.

Another part of the assignment was dealing with class imbalance. The dataset had more customers who did not churn than customers who did churn. This can be a problem because a model may learn to mostly predict the larger class and still appear accurate. I used `class_weight='balanced'` in the Logistic Regression model so the smaller churn class would receive more attention during training. I liked this method because it did not manually add, remove, or change rows in the dataset. Instead, it changed how much importance the model gave to each class.

One challenge was making sure the steps happened in the correct order. The professor instructed us to split the dataset first, perform preprocessing on the training data, and leave the original testing data untouched. My section depended on the earlier cleaning and train-test split being completed before I could begin. I had to make sure I used a copy of the training set instead of accidentally changing the original test data. This helped me understand why data leakage and proper workflow order matter. A model can look more accurate than it really is if information from the test set is used during preprocessing or training.

I also had to think carefully about which columns should be encoded and which columns should be scaled. Not every numeric-looking column should automatically be scaled, and categorical columns need a different type of preparation. This project made the difference between data types feel more practical than it did when we only discussed them in class.

My main contribution to the group was preparing the training data so it became cleaner, numeric, scaled, and ready for machine learning. I simplified repeated categories, one-hot encoded the text columns, scaled the selected numerical features, created before-and-after visualizations, saved the processed training dataset, and added class balancing to the Logistic Regression model. I also wrote an explanation of how these steps connect to the customer churn problem.

Overall, this project helped me see how the concepts from the Exploratory Data Analysis and Data Preparation modules fit together. EDA helps us understand what is happening in a dataset, while preprocessing turns that understanding into actual changes that make the data usable for modeling. My biggest takeaway is that the quality of the model depends heavily on how the data is prepared. Even a good algorithm will struggle if the data is inconsistent, unbalanced, or still contains text values that it cannot understand.